From Fission to Computation: A Mathematical Analogy Between Nuclear Chain Reactions and Agentic AI Compute Criticality

Sivamani Battala

Independent Researcher

October 2025

Abstract

This paper formalizes an analogy between nuclear fission chain reactions and recursive agentic compute propagation in modern AI systems. We present full derivations for nuclear reactor kinetics (including prompt and delayed neutron theory and the Inhour equation), then mirror every step with an equivalent agentic-compute formalism (branching factor, latency, delayed activations). We extend deterministic ODEs to stochastic branching-process theory (Galton–Watson, Bellman–Harris), derive exact and asymptotic formulas for expectations and variances, propose experiment designs for measuring compute criticality, and develop control-theoretic models (logistic feedback) for safe governance. Finally, we generalize to coupled multi-agent networks and give a spectral (eigenvalue) criterion for global compute criticality. The result is a rigorous, measurable framework: *AI Compute Criticality*.

Contents

1 Introduction **4**

2 Part I — Nuclear Chain Reaction Dynamics **4**

2.1 1.1 Basic variables and physical meaning 5

2.2 1.2 Discrete-generation viewpoint and continuous limit 5

2.3 1.3 Solution and regimes 6

2.4 1.4 Prompt vs delayed neutrons: point kinetics equations 6

2.5 1.5 The Inhour equation (reactor period) 7

2.6 1.6 Example numeric illustration (prompt-only) 8

3 Part II — Agentic AI Compute Chain: deterministic mapping **8**

3.1 2.1 Variables and mapping 8

3.2 2.2 Discrete to continuous derivation 9

3.3 2.3 Delayed compute dynamics (precursor analogy) 10

3.4 2.4 Compute Inhour equation 10

3.5 2.5 Interpretation and practical meaning 11

4 Part III — Experimental design and measurement framework **11**

4.1 3.1 High-level experimental logic 11

4.2 3.2 GFLOPs / compute estimation (practical) 12

4.3 3.3 Required logged fields (minimum) 13

4.4 3.4 Estimators and analysis recipes 13

4.5 3.5 Example empirical observations (illustrative) 14

4.6 3.6 Statistical power and practical considerations 14

5 Part IV — Stochastic theory and continuous-time branching processes **15**

5.1 4.1 Galton–Watson branching process (discrete generations) 15

5.2 4.2 Total progeny T and its moments 15

5.3 4.3 Continuous-time branching: Bellman–Harris 16
5.4 4.4 Extinction probability and critical tails 16
5.5 4.5 Early-warning indicators (analytic form) 17

6 Part V — Controlled compute dynamics: feedback and governance 17

6.1 5.1 Feedback model for effective branching 17
6.2 5.2 Controlled ODE and logistic form 18
6.3 5.3 Stability and linearization 18
6.4 5.4 Control metrics and governance knobs 18
6.5 5.5 Control efficiency 19

7 Part VI — Global compute criticality: coupled networks and critical mass 19

7.1 6.1 Coupled-node linear model 19
7.2 6.2 Spectral criticality criterion 20
7.3 6.3 Critical mass and scaling 20
7.4 6.4 Implications 20

8 Conclusions, implications and recommendations 21

8.1 7.1 Summary of mathematical results 21
8.2 7.2 What this means practically 21
8.3 7.3 Experimental checklist for future work 22
8.4 7.4 Final synthesis — a short closing statement 22

1 Introduction

A nuclear chain reaction arises when neutrons released by fission events induce further fissions. If each generation reproduces, on average, more than one neutron that causes fission, the process becomes self-sustaining or explosive. Nuclear reactor physics developed a compact, rigorous mathematical formalism that led to safe, controlled energy production as well as to an understanding of explosive regimes.

In modern agentic AI systems, an inference call (an LLM call, tool call, or sub-agent invocation) may spawn additional calls. When this branching is recursive and unchecked, total computation (FLOPs and tokens) used for a single task may grow dramatically. The branching process formalism that underlies nuclear kinetics can be mapped to agentic compute. This paper develops that mapping thoroughly and provides derivations, stochastic theory, experimental design, and control models.

This document is organized in six main parts:

Part I: Nuclear Chain Reaction Dynamics (complete derivation and kinetics).

Part II: Agentic AI Compute Chain (direct mapping and deterministic derivations).

Part III: Experiment design and empirical measurement logic (how to test compute criticality).

Part IV: Stochastic branching theory and continuous-time (Galton–Watson, Bellman–Harris).

Part V: Controlled compute dynamics (feedback, logistic stabilization).

Part VI: Global compute criticality (coupled-agent networks) and a unified conclusion.

2 Part I — Nuclear Chain Reaction Dynamics

This section re-derives nuclear chain reaction kinetics from first principles, carefully defining variables and showing each algebraic step. This is classical reactor kinetics (prompt +

delayed neutrons) presented in a manner that is precise yet compact.

2.1 1.1 Basic variables and physical meaning

Define:

- $n(t)$ — neutron population (neutrons per unit volume) at time t [neutrons/cm^3].
- k_{eff} — effective neutron multiplication factor (dimensionless); mean number of neutrons in generation g+1 per neutron in generation g.
- l — mean prompt neutron lifetime (s): the average time between birth and absorption/escape.
- ρ — reactivity (dimensionless): $\rho = \dfrac{k_{\text{eff}} - 1}{k_{\text{eff}}}$.
- β — total delayed neutron fraction (dimensionless), and β_i fraction per precursor group i.
- λ_i — decay constant of delayed precursor group i (s^{-1}).
- $C_i(t)$ — concentration (per unit volume) of delayed precursor group i.

2.2 1.2 Discrete-generation viewpoint and continuous limit

Consider discrete generations separated by mean time l. If after one generation the neutron population multiplies by k_{eff}:

$$n_{g+1} = k_{\text{eff}}\, n_g.$$

After g generations (time $t = gl$),

$$n(t) = n(0)\, k_{\text{eff}}^{t/l}.$$

Take natural logarithm:

$$\ln n(t) = \ln n(0) + \frac{t}{l} \ln k_{\text{eff}}.$$

Differentiate with respect to t:

$$\frac{1}{n(t)} \frac{dn(t)}{dt} = \frac{1}{l} \ln k_{\text{eff}},$$

so the exact continuous-time ODE is

$$\boxed{\frac{dn(t)}{dt} = \frac{\ln k_{\text{eff}}}{l} n(t).} \tag{1}$$

For small deviations near unity ($k_{\text{eff}} \approx 1$) use $\ln k_{\text{eff}} \approx k_{\text{eff}} - 1$, giving the standard linearized kinetics:

$$\boxed{\frac{dn(t)}{dt} = \frac{k_{\text{eff}} - 1}{l} n(t).} \tag{2}$$

2.3 1.3 Solution and regimes

Solving (2) yields

$$n(t) = n(0) \exp\left(\frac{k_{\text{eff}} - 1}{l} t \right).$$

Thus:

- $k_{\text{eff}} < 1$ (subcritical): $n(t)$ decays exponentially.
- $k_{\text{eff}} = 1$ (critical): $n(t)$ constant (steady chain).
- $k_{\text{eff}} > 1$ (supercritical): $n(t)$ grows exponentially (runaway).

2.4 1.4 Prompt vs delayed neutrons: point kinetics equations

Delayed neutrons are produced by the decay of precursors; although a small fraction (β) they significantly slow the dynamics and make control possible. The standard point-kinetics equations are:

$$\frac{dn}{dt} = \frac{\rho - \beta}{l} n + \sum_{i=1}^{m} \lambda_i C_i(t), \tag{3}$$

$$\frac{dC_i}{dt} = \frac{\beta_i}{l} n - \lambda_i C_i(t), \qquad i = 1, \ldots, m. \tag{4}$$

Derivation sketch: prompt fissions contribute $(1-\beta)$ fraction immediately; delayed neutrons are produced by precursor decay at rate $\lambda_i C_i$; precursors are produced at rate $\beta_i n/l$ per group.

2.5 1.5 The Inhour equation (reactor period)

Assume exponential solutions $n(t) = n_0 e^{t/\tau}$ and $C_i(t) = C_{i0} e^{t/\tau}$. Substitute into (3)–(4):

From (4):

$$\left(\frac{1}{\tau} + \lambda_i\right) C_{i0} = \frac{\beta_i}{l} n_0 \quad \Rightarrow \quad C_{i0} = \frac{\beta_i/l}{1/\tau + \lambda_i}\, n_0.$$

Plug into (3):

$$\begin{aligned} \frac{1}{\tau} n_0 &= \frac{\rho - \beta}{l} n_0 + \sum_{i=1}^{m} \lambda_i C_{i0} \\ \Rightarrow \frac{1}{\tau} &= \frac{\rho - \beta}{l} + \sum_{i=1}^{m} \frac{\beta_i \lambda_i / l}{1/\tau + \lambda_i}. \end{aligned}$$

Multiply by l and rearrange to obtain the Inhour equation:

$$\boxed{\rho = \frac{l}{\tau} + \sum_{i=1}^{m} \frac{\beta_i}{1 + \lambda_i \tau}.}$$

This equation relates reactivity ρ to the reactor period τ and is the key to control analyses.

2.6 1.6 Example numeric illustration (prompt-only)

Take $n(0) = 10^6$, $k_{\text{eff}} = 1.05$, $l = 1 \times 10^{-4}\,\text{s}$. Then using the linearized form:

$$a = \frac{k_{\text{eff}} - 1}{l} = \frac{0.05}{10^{-4}} = 500\ \text{s}^{-1}.$$

Solution:

$$n(t) = 10^6 e^{500t}.$$

At $t = 0.01\,\text{s}$, $n(0.01) = 10^6 e^5 \approx 1.484 \times 10^8$. This dramatic growth illustrates why prompt supercriticality is explosive on microsecond–millisecond timescales.

Safety note: The above mathematics is standard reactor-physics theory. It quantifies population dynamics and timescales; this exposition intentionally avoids any construction-level or material-level guidance. The goal is mathematical analogy and control-theoretic insight.

3 Part II — Agentic AI Compute Chain: deterministic mapping

We now map nuclear variables into agentic-compute analogues and derive identical-form ODEs that describe expected compute propagation.

3.1 2.1 Variables and mapping

Define the primary quantities and the analogy:

Nuclear variable	AI variable	Meaning
$n(t)$	$C(t)$	Active compute threads (concurrent LLM/tool invocations)
k_{eff}	b	Branching factor (expected children per call)
l	Λ	Mean generation time / latency (s)
ρ	ρ_c	Compute reactivity (departure from criticality)
β_i, λ_i, C_i	β_i, μ_i, D_i	Delayed-call fractions, activation rates, queued tasks
GFLOPs, tokens	GFLOPs, tokens	Compute cost and accounting

3.2 2.2 Discrete to continuous derivation

If each generation multiplies by factor b, then after time t (with generation time Λ):

$$C(t) = C(0)\, b^{t/\Lambda}.$$

Taking logs and differentiating, exact continuous-time equation analogous to (1) is:

$$\frac{dC(t)}{dt} = \frac{\ln b}{\Lambda}\, C(t).$$

Near $b \approx 1$, approximate $\ln b \approx b - 1$ to get the linearized agentic compute ODE:

$$\boxed{\frac{dC(t)}{dt} = \frac{b-1}{\Lambda}\, C(t).} \tag{5}$$

Solution:

$$C(t) = C(0) \exp\!\left(\frac{b-1}{\Lambda} t\right).$$

Thus:

- $b < 1$: compute usage decays — recursion collapses.
- $b = 1$: critical compute — self-sustaining chains, power-law tails expected.

- $b > 1$: supercritical compute — runaway recursion, exponential compute growth.

3.3 2.3 Delayed compute dynamics (precursor analogy)

In realistic agentic systems a fraction of spawned tasks are delayed (e.g., queued workflows, scheduled tool calls, long-running async subtasks). Mirror the point-kinetics form:

$$\frac{dC}{dt} = \frac{\rho_c - \beta}{\Lambda} C + \sum_{i=1}^{m} \mu_i D_i(t), \tag{6}$$

$$\frac{dD_i}{dt} = \frac{\beta_i}{\Lambda} C - \mu_i D_i(t), \qquad i = 1, \ldots, m, \tag{7}$$

where:

- $D_i(t)$ — queued/delayed tasks of type i (calls scheduled to activate later).
- μ_i — activation (decay) rate for delayed tasks type i (s^{-1}).
- β_i — fraction of newly spawned tasks that go into delayed class i.
- ρ_c — compute reactivity: e.g. $\rho_c = \dfrac{b-1}{b}$ or simply $(b-1)$ depending on normalization.

3.4 2.4 Compute Inhour equation

Assuming exponential solutions $C(t) = C_0 e^{t/\tau_c}$ and $D_i(t) = D_{i0} e^{t/\tau_c}$, analogous algebra gives:

$$\boxed{\rho_c = \frac{\Lambda}{\tau_c} + \sum_{i=1}^{m} \frac{\beta_i}{1 + \mu_i \tau_c}.}$$

This compute-Inhour relation gives the compute period τ_c (time for compute to grow by factor e) for a given branching/delay configuration.

3.5 2.5 Interpretation and practical meaning

- If $\rho_c < 0$ (equivalently $b < 1$ under a given normalization), recursion dies out rapidly.
- If $0 < \rho_c <$ (delayed fraction), growth is governed by delayed activations (longer timescales).
- If ρ_c exceeds delayed fractions akin to delayed neutron fractions, the system may experience fast, hard-to-control bursts.

4 Part III — Experimental design and measurement framework

To empirically demonstrate compute criticality we must (A) design prompt/control sweeps that change b, (B) log parent-child relationships and per-call costs, and (C) analyze using branching-process statistics.

This section provides a rigorous, publication-ready experimental protocol and the analytic measurements needed to estimate b, detect criticality signatures, and quantify amplification.

4.1 3.1 High-level experimental logic

1. Fix a moderately challenging user task inviting planning and web lookups (e.g., "Summarize X with citations, extract facts, draft an email").
2. Create prompt-policy variants that monotonically increase permission to spawn helpers: P0 (no spawning), P1/P2 (limited helper count), P6 (full spawn freedom).
3. For each prompt variant run R independent trials (random seeds where available).
4. Instrument every LLM/tool call with:
    - unique call_id and parent_id,

- model/provider and settings,
- tokens in and out,
- start and end timestamps (latency),
- per-call GFLOPs estimate (see below),
- content preview,
- termination flag (budget abort).

5. Aggregate logs per trial and compute trial-level metrics, then perform statistical analysis across trials/variants.

4.2 3.2 GFLOPs / compute estimation (practical)

Exact GFLOPs per call depends on model architecture and are often unavailable. A pragmatic estimator:

$$\text{GFLOPs}_{\text{call}} \approx \text{token_count} \times \text{model_flop_scale} \times \alpha,$$

where:

- token_count = tokens_in + tokens_out,
- model_flop_scale — a rough per-token FLOP count (depends on model size; e.g., 1e12 for large multi-trillion parameter families, adjust for smaller),
- α — unit conversion to GFLOPs (e.g., 6×10^{-9} if model_flop_scale is FLOPs per token).

This formula is a heuristic for cross-trial comparison; absolute FLOPs require vendor data.

4.3 3.3 Required logged fields (minimum)

Per-call CSV/JSON line must include:

(timestamp, trial_id, call_id, parent_id, provider, model, tokens_in, tokens_out, latency_s, GFLOPs, con

4.4 3.4 Estimators and analysis recipes

Offspring counts and b estimation. Within each trial, compute child counts per call by counting how many calls list a given call's id as parent. Then estimate

$$\hat{b}_{\text{trial}} = \frac{1}{N_{\text{calls}}} \sum_{\text{call } i} \text{children}_i.$$

Aggregate across trials of a variant to get $\hat{b}_{\text{variant}}$ and bootstrap 95% CIs by resampling calls (or trials) as appropriate.

Amplification factor. Choose P0 as baseline (single-pass). Let F_0 be mean total GFLOPs for P0. For any trial compute amplification:

$$A = \frac{F_{\text{trial}}}{F_0}.$$

Plot A vs $\hat{b}$ (log scale on A), expecting large increases as $\hat{b}$ approaches 1 and exceeds 1.

Heavy-tail detection. Aggregate trial total calls T. For each variant estimate CCDF and test for power-law tails (Clauset–Shalizi–Newman method): fit a tail exponent α for $P(T \geq t) \propto t^{-(\alpha-1)}$ and compute goodness-of-fit (KS test against fitted power law). At criticality ($b \approx 1$) theory predicts heavy tails.

Early-warning statistics. Compute across-trial variance of total GFLOPs and compute autocorrelation at lag-1 of $C(t)$ (if you have fine-grained time series). Theory predicts:

$$\mathrm{Var}(T) \propto (1-b)^{-3}, \qquad \tau_{\mathrm{recover}} \propto \frac{1}{1-b}.$$

4.5 3.5 Example empirical observations (illustrative)

From runs you provided as a PoC (sample lines reproduced in logs): trials across P0, P3, P6 show:

- P0: total calls observed $\in \{1, \ldots, 10\}$; total GFLOPs typically $\sim 10^7$–10^8 (baseline).
- P3: more frequent multi-call behavior; some trials with 9–10 calls and GFLOPs $\sim 10^8$, others modest.
- P6: broader variance, some trials showing tens of millions to $\sim 10^8$ GFLOPs and more calls; occasional deep helper spawning.

Qualitatively: small prompt permissiveness changes produced non-linear increases in total compute in some trials — consistent with moving b nearer or above unity.

4.6 3.6 Statistical power and practical considerations

To estimate b stably near 1, you need many trials because variance diverges near criticality. We recommend:

- $R \geq 50$ independent trials per variant for publication-quality CI estimates,
- granular logging (parent-child) to compute true offspring counts,
- simulate a controlled "fake" helper response strategy to test sensitivity without incurring real cost.

5 Part IV — Stochastic theory and continuous-time branching processes

This section develops the rigorous stochastic underpinning that justifies the ODE mean-field forms, and provides formulas for expectations, variances, extinction probabilities and heavy-tail asymptotics.

5.1 4.1 Galton–Watson branching process (discrete generations)

Let X be the random number of children produced by a single call; $P(X = k) = p_k$, $k \geq 0$. Define C_n the population size at generation n. Then:

$$C_{n+1} = \sum_{i=1}^{C_n} X_i$$

with X_i i.i.d. copies of X. The offspring generating function is

$$f(s) = \mathbb{E}[s^X] = \sum_{k=0}^{\infty} p_k s^k.$$

Key results:

- Mean: $\mathbb{E}[C_n] = b^n C_0$, where $b = \mathbb{E}[X] = f'(1)$.
- Extinction probability q is the smallest non-negative root of $q = f(q)$.
- If $b \leq 1$, $q = 1$; if $b > 1$, $q < 1$ (positive probability of infinite growth).

5.2 4.2 Total progeny T and its moments

Total calls across all generations:

$$T = \sum_{n=0}^{\infty} C_n.$$

When $b < 1$,

$$\mathbb{E}[T] = \frac{C_0}{1-b}.$$

The variance (for Galton–Watson with finite variance σ^2 of X) scales as:

$$\boxed{\mathrm{Var}(T) \approx \frac{\sigma^2 C_0}{(1-b)^3}, \quad (b \to 1^-).}$$

This cubic divergence is a hallmark of critical slowing and variance explosion.

5.3 4.3 Continuous-time branching: Bellman–Harris

When individual lifetimes are random (latency distribution G) and births occur at death, the Bellman–Harris process governs dynamics. Let $C(t)$ be active population at time t. The mean $M(t) = \mathbb{E}[C(t)]$ satisfies renewal-type integral equation:

$$M(t) = C_0 \bar{G}(t) + \int_0^t b\, M(t-\tau)\, dG(\tau),$$

where $\bar{G}(t) = 1 - G(t)$.

For exponential latency ($G(\tau) = 1 - e^{-\lambda\tau}$), differentiate to get:

$$\frac{dM}{dt} = \lambda(b-1)M(t),$$

recovering the mean-field ODE (5) with $\lambda = 1/\Lambda$.

5.4 4.4 Extinction probability and critical tails

At criticality ($b = 1$), the tail of the total progeny distribution follows a universal power law:

$$P(T \geq t) \sim ct^{-1/2}, \qquad \text{or } P(T = n) \sim Cn^{-3/2},$$

depending on normalization. This implies rare but arbitrarily large runs — consistent with observed heavy-tailed agentic sessions.

5.5 4.5 Early-warning indicators (analytic form)

As $b \to 1^-$:

- Mean total workload $\mathbb{E}[T] \sim (1-b)^{-1}$.
- Variance $\mathrm{Var}(T) \sim (1-b)^{-3}$.
- Recovery time (autocorrelation decay) scales $\tau_{\text{recover}} \sim (1-b)^{-1}$.

Therefore measuring increases in variance, autocorrelation and tail-heaviness in logged compute traces provides diagnostics for approaching criticality.

6 Part V — Controlled compute dynamics: feedback and governance

We now introduce feedback mechanisms that dynamically reduce the effective branching factor as compute grows (the analogue of control rods and moderators). This produces a logistic-like stabilization.

6.1 5.1 Feedback model for effective branching

Let b_0 be intrinsic branching (uncontrolled). Implement negative feedback based on instantaneous compute:

$$b_{\text{eff}}(t) = b_0 \left(1 - \gamma \frac{C(t)}{C_{\max}}\right), \tag{8}$$

with $0 \le \gamma \le 1$ and $C_{\max}$ a design budget.

6.2 5.2 Controlled ODE and logistic form

Plugging (8) into (5) (with $\lambda = 1/\Lambda$) yields:

$$\begin{aligned}\frac{dC}{dt} &= \lambda\,(b_{\text{eff}}(t) - 1)\,C(t) \\ &= \lambda\Big(b_0 - 1 - b_0\gamma\frac{C(t)}{C_{\max}}\Big)C(t).\end{aligned}$$

Define $\alpha = \lambda(b_0 - 1)$ and $\beta = \lambda b_0\gamma/C_{\max}$, we have

$$\boxed{\frac{dC}{dt} = \alpha C - \beta C^2.}$$

This is the logistic ODE. Its solution (for $\alpha \neq 0$) is:

$$C(t) = \frac{C_{\text{eq}}}{1 + \left(\frac{C_{\text{eq}}}{C(0)} - 1\right)e^{-\alpha t}}, \qquad C_{\text{eq}} = \frac{\alpha}{\beta} = \frac{(b_0 - 1)C_{\max}}{b_0\gamma}.$$

6.3 5.3 Stability and linearization

Linearize around equilibrium C_{eq} with $\delta C(t) = C(t) - C_{\text{eq}}$:

$$\frac{d(\delta C)}{dt} \approx -\alpha\,\delta C,$$

so perturbations decay with time constant $\tau_{\text{stab}} = 1/\alpha = \dfrac{\Lambda}{\lambda(b_0 - 1)}$ (note parameter normalization). Stronger feedback γ reduces C_{eq} and shortens transient excursions (via β).

6.4 5.4 Control metrics and governance knobs

Analogues of reactor controls:

- **Throttle/limit (control rod):** reduce spawn permissions (lower b_0 effectively).
- **Budget cap $C_{\max}$:** hard budget limit, saturates growth.

- **Dynamic throttling γ:** continuous feedback strength.
- **Latency buffers:** increase Λ (reduce λ) slowing growth rate.
- **Scram:** emergency global abort, immediate $C(t) \to 0$.

6.5 5.5 Control efficiency

Define long-term suppression ratio:

$$\eta_c = \frac{C_{\text{eq}}}{C_{\text{uncontrolled}}(t \to \infty)}.$$

With uncontrolled exponential growth $C_{\text{uncontrolled}}(t)$ diverging, $\eta_c \to 0$ indicates successful suppression. In practice, evaluate η_c at finite horizons and under stochastic noise.

7 Part VI — Global compute criticality: coupled networks and critical mass

The single-agent model generalizes to a network of interacting agents. Inter-agent coupling can produce a global critical transition even if individual nodes are (locally) subcritical.

7.1 6.1 Coupled-node linear model

Let N nodes with local compute $C_i(t)$, intrinsic branching b_i and latency Λ_i. Let $\kappa_{ij} \geq 0$ be coupling (calls from j stimulating i). Include local dissipation/governance δ_i:

$$\frac{dC_i}{dt} = \frac{b_i - 1}{\Lambda_i} C_i + \sum_{j \neq i} \kappa_{ij} C_j - \delta_i C_i.$$

Matrix form: $\dot{\mathbf{C}} = \mathbf{AC}$ with

$$A_{ii} = \frac{b_i - 1}{\Lambda_i} - \delta_i, \qquad A_{ij} = \kappa_{ij} \ (i \neq j).$$

7.2 6.2 Spectral criticality criterion

Let $\lambda_{\max}$ be largest real part eigenvalue of $\mathbf{A}$. Then:

$$\lambda_{\max} < 0 \Rightarrow \text{global decay (subcritical)}, \quad \lambda_{\max} > 0 \Rightarrow \text{global growth (supercritical)}.$$

Hence the network is critical when $\lambda_{\max} = 0$. This is the exact analogue of critical mass: connectivity and coupling can push a system over the threshold.

7.3 6.3 Critical mass and scaling

Define global compute mass $M_c = \sum_i C_i$. Under simplifying homogeneous assumptions ($b_i = b$, $\Lambda_i = \Lambda$, $\delta_i = \delta$ and symmetric average coupling $\bar{\kappa}$) one can derive an effective net branching:

$$b_{\text{net}} \approx 1 + \frac{\Lambda}{N}\Big(\sum_i \frac{b-1}{\Lambda} + \sum_{i \neq j} \kappa_{ij} - \sum_i \delta_i\Big).$$

Criticality occurs when $b_{\text{net}} = 1$. Intuitively, adding more agents, increasing coupling κ, or lowering damping δ can shift the whole network to ignition.

7.4 6.4 Implications

- Localised governance is insufficient: global coupling must be considered.
- Systems-of-systems risk: many marginally autonomous services lightly coupled can produce emergent compute bursts.
- Monitoring and spectral analyses of adjacency-weighted compute matrices can provide early warnings.

8 Conclusions, implications and recommendations

8.1 7.1 Summary of mathematical results

We have shown:

1. Both nuclear chain reactions and recursive agentic compute follow mathematically equivalent branching dynamics; their mean-field equations are identical in form.
2. Deterministic ODE: $\frac{dX}{dt} = \frac{R-1}{\tau} X$ (neutrons or compute threads).
3. Stochastic branching theory predicts divergence of variance near criticality, heavy-tailed total work distributions, extinction probabilities, and universal power-law tails at $R = 1$.
4. Feedback (control) converts exponential growth into logistic saturation if properly designed.
5. Network coupling introduces global criticality via the dominant eigenvalue of the linearized coupling matrix.

8.2 7.2 What this means practically

- **Measurability:** Instrumentation (parent-child linkages, tokens, latencies) enables empirical estimates of b and detection of criticality precursors (variance explosion, critical slowing).
- **Governance:** Design levers analogous to reactor controls (throttling, budget caps, latency buffers, emergency aborts) mathematically stabilize compute if tuned to keep $b_{\text{eff}} < 1$ or $\lambda_{\max} < 0$ for networks.
- **System design:** Architectures should limit autonomous spawn privileges by default and require explicit, auditable elevation for broader autonomy.

- **Network risk:** Large distributed agent ecosystems require system-level spectral monitoring to preempt collective ignition.

8.3 7.3 Experimental checklist for future work

To build on this paper, an ideal empirical program would:

1. Implement the experiment design in Part III with $R \geq 50$ trials per variant.
2. Ensure strict parent_id logging to measure offspring distributions exactly.
3. Fit branching distributions p_k, estimate $\hat{b}$ and $\hat{\sigma}^2$ with bootstrap CIs.
4. Test early-warning indicators as control parameters vary (e.g., reward shaping, tool availability).
5. Validate logistic-control models by simulating/implementing feedback and measuring C_{eq} and τ_{stab}.

8.4 7.4 Final synthesis — a short closing statement

Mathematically, both neutron chains and agentic compute obey the same simple tension: reproduction vs loss. When reproduction exceeds unity, systems spontaneously self-amplify; when it is less than unity, activity collapses. Between these is a knife-edge: criticality. Armed with the mathematical toolkit of branching processes, reactor kinetics and control theory, we can quantify, measure, and govern the transition for agentic AI — but only if we instrument our systems and design governance with the same mathematical seriousness used in reactor design.

Epilogue. In 1942, Fermi created the first controlled self-sustaining nuclear chain reaction — not because the math was unknown, but because the theoretical framework was connected to careful measurement, instrumentation and control. If agentic AI systems are to be

beneficial rather than hazardous, we must bring equivalent rigor: measurement of b, logging of parent-child links, spectral monitoring of networks, and mathematically principled governors. Criticality is not merely a metaphor: it is a dynamical transition we can and must treat quantitatively.

Acknowledgements

The author thanks colleagues and early reviewers who discussed branching-process analogies with agentic systems. Any errors are the author's.

References (selected)

- J. R. Harris, *The Theory of Branching Processes*, Dover (2002 reprint).
- J. R. Lamarsh, *Introduction to Nuclear Reactor Theory*, Addison-Wesley.
- J. Glasstone & A. Sesonske, *Nuclear Reactor Engineering*, Springer.
- Bellman, R. & Harris, T. E., "Branching Processes", Bulletin of the American Mathematical Society (1952).
- Clauset, A., Shalizi, C. R., Newman, M. E. J., "Power-Law Distributions in Empirical Data", SIAM Review (2009).
- Kaplan, J., McCandlish, S., Henighan, T., et al., "Scaling Laws for Neural Language Models", (Representative modern work on compute scaling).
- Battala, S., "Agentic AI Compute Criticality: Experimental Observation of Recursive Scaling Transitions", Working Paper (2025).